Canterbury Christ Church University

Assessment 1 Advanced OS

SPACES

ASSIGNMENT GUIDELINES

TITLE OF MODULE:
Advanced Operating Systems U14553

MODULE COMPONENT:
50% of Module

MODULE TEAM:
Dr Victor Obarafor
Tim Jackson

ASSIGNMENT CONTACT:
Dr Victor Obarafor
Victor.obarafor@canterbury.ac.uk

ASSESSMENT TYPE:
Report + Artefact

WHERE TO SUBMIT:
BLACKBOARD TURNITIN SUBMISSION TOOL

DUE DATE: 23rd March 2PM

If you experience any problems with this system, please contact the Computing Administration Team.

WHAT TO SUBMIT:
An 800 word maximum report of high professional standard + Artefact source code

ASSIGNMENT INSTRUCTIONS

This assessment evaluates your ability to apply operating system management techniques and automation tools in a real-world context. It is structured into three scripting-based tasks requiring you to write Bash or Python scripts to manage processes, schedule jobs, validate files, and implement automated system controls.

1. University Data Centre Process and Resource Management System
Bash Scripting Only

This task aligns with Learning Outcome 2 and demonstrates system automation techniques for process monitoring, disk management, and log control, which are key tools for operating system administration.

The University Data Centre hosts multiple research and academic services on shared Linux servers. Performance degradation has been observed due to unmanaged background processes, excessive memory consumption, and uncontrolled growth of log files.

Your task is to write a Bash script that simulates an intelligent system administration tool capable of executing the following operations via a menu driven interface.

Requirements

1. Process Monitoring and Management
   • Display current CPU and memory usage.
   • List the top ten memory consuming processes including PID, user, CPU percentage, and memory percentage.
   • Terminate a selected process only after confirmation.
   • Prevent termination of critical system processes.
2. Disk Inspection and Log Archiving
   • Inspect disk usage of a specified directory.
   • Detect log files larger than 50MB.
   • Create an ArchiveLogs directory if it does not exist.
   • Compress large log files into ArchiveLogs with a timestamped filename.
   • Display a warning if ArchiveLogs exceeds 1GB.
3. Logging System
   • Maintain a system_monitor_log.txt file.
   • Record all administrative actions with timestamps.
4. Exit Mechanism
   • Implement a Bye exit command with a Y or N confirmation prompt.

2. University High Performance Computing Job Scheduler
Bash or Python

This task aligns with Learning Outcome 4 which requires using Bash or Python scripting to implement scheduling algorithms and manage queue systems efficiently within a POSIX compliant environment.

The University High Performance Computing Laboratory provides limited graphical processing resources to research students. Manual allocation has resulted in unfair scheduling and inefficient resource usage.

You are required to develop a script that manages computational job requests and processes them using Round Robin or Priority Scheduling.

Requirements

1. User Menu
   • View pending jobs.
   • Submit a job request.
   • Process job queue using either Round Robin or Priority scheduling.
   • View completed jobs.
   • Exit system with confirmation.
2. Queue Based Scheduling System
   • Each job must include student ID, job name, estimated execution time, and priority value between 1 and 10.
   • Round Robin must use a time quantum of five seconds.
   • Priority scheduling must execute highest priority jobs first.
3. Logging System
   • Log every job submission and execution event in scheduler_log.txt.
   • Record student ID, job name, scheduling type, and timestamp.
4. Data Storage
   • Store pending jobs in job_queue.txt.
   • Store completed jobs in completed_jobs.txt.

3. Secure Examination Submission and Access Control System
Bash and Python

This task aligns with Learning Outcomes 2 and 4. It demonstrates system administration techniques for file validation, duplicate detection, logging mechanisms, and security monitoring.

The Examination Board requires a secure submission system that validates file formats, prevents duplicate submissions, and detects suspicious login behaviour.

You must develop a script that validates student submissions and monitors login attempts.

Requirements

1. Menu System
   • Submit an assignment.
   • Check if a file has already been submitted.
   • List all submitted assignments.
   • Simulate login attempt.
   • Exit system with confirmation.

2. File Validation
   • Only accept .pdf and .docx files.
   • Reject files larger than 5MB.
3. Duplicate Submission Detection
   • Reject submissions with identical filename and content.
   • Log all submissions in submission_log.txt.
4. Access Control and Suspicious Activity Detection
   • Lock accounts after three failed login attempts.
   • Detect repeated login attempts within sixty seconds.
   • Log all login attempts with timestamps.

REPORT WRITE UP
800 Words.

Alongside your source code, you must submit an 800-word report that explains your implementation in detail.

The report should

1. Explain your design choices and justify command selection.
2. Connect your implementation to advanced operating system concepts such as process management, scheduling algorithms, file system operations, and automation techniques.
3. Include at least three academic or technical references using Harvard referencing.
4. Identify challenges encountered and how they were resolved.
5. Provide execution steps and expected output with screenshots.

MARKING SCHEME

Task 1 Process and Resource Management System 15
Task 2 High Performance Computing Scheduler 15
Task 3 Secure Submission and Access Control System 15
Report Write up 5

TOTAL 50

ASSESSMENT RUBRIC

Each task is graded out of 15 marks while the report is worth 5 marks, making a total of 50 marks.

The final grade is determined by the sum of all tasks and the report.

TASK 1
UNIVERSITY DATA CENTRE PROCESS AND RESOURCE MANAGEMENT SYSTEM
15 MARKS

CRITERIA	MARKS	DESCRIPTION
MENU SYSTEM AND USER INTERACTION	3	The script provides a complete interactive menu. All required administrative options are available. Invalid inputs are handled appropriately without system failure.
PROCESS MONITORING AND MANAGEMENT	4	Processes are displayed correctly including PID, user, CPU percentage, and memory percentage. Safe termination is implemented with confirmation. Critical processes are protected from termination.
DISK INSPECTION AND LOG ARCHIVING	4	Large log files are detected correctly. ArchiveLogs directory is created if required. Files are compressed with timestamps. Warning is displayed when storage threshold exceeds 1GB.
LOGGING SYSTEM	2	All administrative actions are recorded in system_monitor_log.txt with timestamps and clear descriptions.
EXIT MECHANISM AND CONFIRMATION	2	A Bye command is implemented and requires Y or N confirmation before termination.

TASK 2
UNIVERSITY HIGH PERFORMANCE COMPUTING JOB SCHEDULER
15 MARKS

CRITERIA	MARKS	DESCRIPTION
MENU SYSTEM AND USER INTERACTION	3	The system provides complete navigation options including job submission, queue viewing, execution, and exit. Input validation is implemented correctly.
SCHEDULING ALGORITHM IMPLEMENTATION	4	Round Robin scheduling operates correctly using a five second time quantum. Priority scheduling correctly executes highest priority jobs first. Execution order is clearly displayed.
QUEUE AND DATA STORAGE SYSTEM	4	Jobs are stored correctly in job_queue.txt. Completed jobs are recorded in completed_jobs.txt. Queue ordering is accurate for both scheduling methods.
LOGGING SYSTEM	2	All scheduling events are recorded in scheduler_log.txt including timestamp, student ID, job name, scheduling type, and execution result.
EXIT MECHANISM AND CONFIRMATION	2	A Bye command is implemented and requires confirmation before exiting the system.

TASK 3
SECURE EXAMINATION SUBMISSION AND ACCESS CONTROL SYSTEM
15 MARKS

CRITERIA	MARKS	DESCRIPTION
MENU SYSTEM AND USER INTERACTION	3	The system provides a complete menu and handles invalid inputs appropriately.
FILE VALIDATION	4	Only pdf and docx files are accepted. Files larger than 5MB are rejected. Invalid formats are identified immediately.
DUPLICATE DETECTION AND ACCESS CONTROL	4	The system detects duplicate filenames and identical content. Duplicate submissions are rejected. Accounts are locked after three failed login attempts. Suspicious login patterns are detected.
LOGGING SYSTEM	2	All submission and login events are recorded in submission_log.txt including timestamp, student ID, filename, and status.
EXIT MECHANISM AND CONFIRMATION	2	A Bye command is implemented with Y or N confirmation before termination.

REPORT WRITE UP
5 MARKS

CRITERIA	MARKS	DESCRIPTION
CODE EXPLANATION AND SCREENSHOTS	2	The report clearly explains implementation decisions and includes evidence of script execution.
CHALLENGES AND SOLUTIONS	1	Technical challenges are identified and solutions are explained clearly.
EXECUTION STEPS AND EXPECTED OUTPUT	1	Clear step by step execution instructions are provided along with expected behaviour.
PROFESSIONAL FORMATTING AND REFERENCING	1	The report is well structured, written in academic language, and follows Harvard referencing style.

ADDITIONAL SUBMISSION REQUIREMENTS

GITHUB REPOSITORY

You must maintain a private GitHub repository for this assessment.

Your repository must contain

All source code files
Clear commit history showing development progression
A README file explaining how to execute each script

You must include the GitHub repository link inside your written report.

Failure to provide a repository link may result in mark reduction.

USE OF GENERATIVE ARTIFICIAL INTELLIGENCE

Students must refer to their Programme Handbook for guidance on the permitted use of Generative Artificial Intelligence tools.

Any use of Generative Artificial Intelligence must comply strictly with university academic integrity policies.

Improper or undeclared use of such tools may result in investigation under academic misconduct regulations.

SUBMISSION GUIDELINES

Follow the required compressed file naming calculation as stated in the original instructions.

Failure to submit a written report will result in a zero mark.
Failure to provide adequate write up will result in mark scaling.

LEARNING OUTCOMES ASSESSED

Outcome 2
Exploit tools and techniques that allow administrators to manage the operating system efficiently and critically evaluate their efficacy.

Outcome 4
Demonstrate the ability to competently operate the UNIX command line environment including knowledge of the POSIX standard and advanced shell scripting.